

Лабораторная работа №6. Паттерны проектирования.

Предварительная подготовка.

Пусть имеется иерархия классов «Выражение» → («Число», «Бинарная операция», «Вызов функции», «Переменная»), описанные ниже. Будем в дальнейших заданиях использовать эти классы. Бинарная операция подразумевает возможность выполнения сложения, умножения, вычитания и деления; вызов функции — подразумевает работу с вызовом sqrt (квадратный корень) и abs (модуль). У переменной есть имя (значение пока ей не присваиваем).

```
struct Expression //базовая абстрактная структура
{
    virtual ~Expression() { } //виртуальный деструктор
    virtual double evaluate() const = 0; //абстрактный метод «вычислить»
};

struct Number : Expression // структура «Число»
{
    Number(double value): value_(value) {} //конструктор
    double value() const { return value_; } // метод чтения значения числа
    double evaluate() const { return value_; } // реализация виртуального метода
    «вычислить»
    ~Number() {} //деструктор, тоже виртуальный

private:
    double value_; // само вещественное число
};

struct BinaryOperation : Expression // «Бинарная операция»
{
    enum { // перечислим константы, которыми зашифруем символы операций
        PLUS = '+',
        MINUS = '-',
        DIV = '/',
        MUL = '*'
    };

    // в конструкторе надо указать 2 операнда — левый и правый, а также сам символ
    // операции
    BinaryOperation(Expression const *left, int op, Expression const *right)
        : left_(left), op_(op), right_(right)
    { assert(left_ && right_); }

    ~BinaryOperation() //в деструкторе освободим занятую память
    {
        delete left_;
        delete right_;
    }

    Expression const *left() const { return left_; } // чтение левого операнда
    Expression const *right() const { return right_; } // чтение правого
операнда
    int operation() const { return op_; } // чтение символа операции

    double evaluate() const // реализация виртуального метода «вычислить»
    {
        double left = left_>evaluate(); // вычисляем левую часть
        double right = right_>evaluate(); // вычисляем правую часть
        switch (op_) // в зависимости от вида операции, складываем, вычитаем,
```

```
умножаем
    // или делим левую и правую части
    {
        case PLUS: return left + right;
        case MINUS: return left - right;
        case DIV: return left / right;
        case MUL: return left * right;
    }
}

private:
    Expression const *left_; // указатель на левый операнд
    Expression const *right_; // указатель на правый операнд
    int op_; // символ операции
};

struct FunctionCall : Expression // структура «Вызов функции»
{
    // в конструкторе надо учесть имя функции и ее аргумент
    FunctionCall(std::string const &name, Expression const *arg): name_(name),
    arg_(arg)
    {
        assert(arg_);
        assert(name_ == "sqrt" || name_ == "abs"); // разрешены только вызов
//sqrt и abs
        std::string const & name() const
        { return name_; }

        Expression const *arg() const // чтение аргумента функции
        { return arg_; }

        ~FunctionCall(){delete arg_;} // освобождаем память в деструкторе

        virtual double evaluate() const { // реализация виртуального метода
//«вычислить»
            if (name_=="sqrt")
                return sqrt(arg_->evaluate()); // либо вычисляем корень квадратный
            else return fabs(arg_->evaluate()); // либо модуль – остальные функции
//запрещены
        }

private:
        std::string const name_; // имя функции
        Expression const *arg_; // указатель на ее аргумент
    };

struct Variable : Expression // структура «Переменная»
{
    Variable(std::string const &name) : name_(name) { } //в конструкторе надо
//указать ее имя
    std::string const &name() const { return name_; } // чтение имени переменной
    double evaluate() const // реализация виртуального метода «вычислить»
    { return 0.0; }

private:
    std::string const name_; // имя переменной
};
```

Проверьте работу иерархии, например, на следующих фрагментах кода:

```
//-----  
Expression * e1 = new Number(1.234);  
Expression * e2 = new Number(-1.234);  
Expression * e3 = new BinaryOperation(e1, BinaryOperation::DIV, e2);  
  
cout<<e3->evaluate()<<endl;  
//-----  
Expression* n32 = new Number(32.0);  
Expression* n16 = new Number(16.0);  
Expression* minus = new BinaryOperation(n32, BinaryOperation::MINUS, n16);  
Expression* callSqrt = new FunctionCall("sqrt", minus);  
Expression* n2 = new Number(2.0);  
Expression* mult = new BinaryOperation(n2, BinaryOperation::MUL, callSqrt);  
Expression* callAbs = new FunctionCall("abs", mult);  
  
cout<<callAbs->evaluate()<<endl;  
//-----
```

Для начала, иерархия классов Expression (приведенная выше) представляет абстрактное синтаксическое дерево (АСД) очень простого языка. Мы немного изменим интерфейс базового класса Expression, добавим в него виртуальный метод:

```
virtual Expression * transform(Transformer *tr) const = 0;
```

Этот метод будет возвращать **полностью новое** (т. е. все узлы должны копироваться, даже если никаких изменений в дереве не требуется) АСД, построенное с помощью реализации интерфейса Transformer из исходного дерева. А сам интерфейс Transformer будет выглядеть следующим образом:

```
struct Transformer  
{  
    virtual ~Transformer() {}  
    virtual Expression *transformNumber(Number const *) = 0;  
    virtual Expression *transformBinaryOperation(BinaryOperation const *) = 0;  
    virtual Expression *transformFunctionCall(FunctionCall const *) = 0;  
    virtual Expression *transformVariable(Variable const *) = 0;  
};
```

Все классы вам уже должны быть известны.

Интерфейс Transformer — это реализация паттерна проектирования Visitor, далее мы детально посмотрим на реализацию метода transform для классов иерархии Expression. Рассмотрим реализацию метода transform на примере простого класса Number:

```
Expression * Number::transform(Transformer *tr) const  
{  
    return tr->transformNumber(this);  
}
```

Рассмотрим пример использования этой конструкции:

```
Expression *expression = new Number(10.0);  
Transformer *transformer = new AconcreteTransformer();  
Expression *new_expression = expression->transform(transformer);
```

Обратите внимание, что несмотря на то, что тип переменной expression Expression *, а не Number *, в результате вызова expression->transform(transformer) будет вызван именно метод transform класса Number (потому что метод transform виртуальный), а он, в свою очередь, вызовет метод transformNumber(Number const *) класса Transformer, т. е. мы смогли определить конкретный класс, используя только указатель на Expression.

Реализация метода transform для других классов иерархии Expression идентична с точностью до имени вызываемого метода (transformBinaryOperation, transformFunctionCall и др). Этот трюк называется **двойной диспетчеризацией**, он позволяет отличить конкретный класс некоторой иерархии от других.

C++ предоставляет и более простой способ узнать конкретный класс по указателю (или ссылке) на базовый класс. Для этого можно использовать оператор **dynamic_cast**. Например, если у вас есть указатель Expression *, и вы хотите узнать, указывает ли этот указатель на самом деле на объект Number, то сделать это можно так:

```
Expression *expression = parse(code);
Number *number = dynamic_cast<Number *>(expression);
if (number)
    std::cout << "It's a number" << std::endl;
else
    std::cout << "It is not a number" << std::endl;
```

Если expression действительно указывает на объект Number (или на один из его наследников, но в нашем примере их нет), то оператор dynamic_cast<Number *> вернет правильный указатель. Если expression указывает не на класс Number, то будет возвращен нулевой указатель. Т. е. если в переменной number хранится нулевой указатель, значит expression не указывает на Number на самом деле.

Для правильной работы оператора dynamic_cast в нашем примере требуется, чтобы класс Expression был полиморфным, т. е. в нем должна быть как минимум одна виртуальная функция (например, деструктор). Стандарт определяет работу dynamic_cast и с не полиморфными типами, но в этом случае никаких проверок типа времени исполнения выполняться не будет - часто, это не то, что требуется.

Если передать оператору dynamic_cast нулевой указатель, то он просто вернет нулевой указатель нужного типа, поэтому проверять указатель перед передачей в dynamic_cast не нужно.

Задание 1. (5 баллов) Реализуйте класс CopySyntaxTree, который, используя шаблон Visitor, выполняет копирование AST. Интерфейсы всех используемых классов приведены для удобства, вам надо будет привести их полную реализацию. Заготовка класса CopySyntaxTree находится в самом низу.

```
struct Transformer;
struct Number;
struct BinaryOperation;
struct FunctionCall;
struct Variable;

struct Expression
{
    virtual ~Expression() { }
```

```
virtual double evaluate() const = 0;
virtual Expression *transform(Transformer *tr) const = 0;
};

struct Transformer //pattern Visitor
{
    virtual ~Transformer() { }
    virtual Expression *transformNumber(Number const *) = 0;
    virtual Expression *transformBinaryOperation(BinaryOperation const *) = 0;
    virtual Expression *transformFunctionCall(FunctionCall const *) = 0;
    virtual Expression *transformVariable(Variable const *) = 0;
};

struct Number : Expression
{
    Number(double value);
    double value() const;
    double evaluate() const;
    Expression *transform(Transformer *tr) const;

private:
    double value_;
};

struct BinaryOperation : Expression
{
    enum {
        PLUS = '+',
        MINUS = '-',
        DIV = '/',
        MUL = '*'
    };
    BinaryOperation(Expression const *left, int op, Expression const *right);
    ~BinaryOperation();
    double evaluate() const;
    Expression *transform(Transformer *tr) const;
    Expression const *left() const;
    Expression const *right() const;
    int operation() const;

private:
    Expression const *left_;
    Expression const *right_;
    int op_;
};

struct FunctionCall : Expression
{
    FunctionCall(std::string const &name, Expression const *arg);
    ~FunctionCall();
    double evaluate() const;
    Expression *transform(Transformer *tr) const;
    std::string const &name() const;
    Expression const *arg() const;

private:
    std::string const name_;
    Expression const *arg_;
};
```

```
struct Variable : Expression
{
    Variable(std::string const name);
    std::string const & name() const;
    double evaluate() const;
    Expression *transform(Transformer *tr) const;

private:
    std::string const name_;
};

/**
 * реализуйте все необходимые методы класса
 * вы можете определять любые вспомогательные
 * методы, если хотите
 */
struct CopySyntaxTree : Transformer
{
    Expression *transformNumber(Number const *number)
    {
        // ваш код
    }

    Expression *transformBinaryOperation(BinaryOperation const *binop)
    {
        // ваш код
    }

    Expression *transformFunctionCall(FunctionCall const *fcall)
    {
        // ваш код
    }

    Expression *transformVariable(Variable const *var)
    {
        // ваш код
    }
};
```

Проверьте работу CopySyntaxTree, например, на таком коде:

```
Number* n32 = new Number(32.0);
Number* n16 = new Number(16.0);
BinaryOperation* minus = new BinaryOperation(n32, BinaryOperation::MINUS, n16);
FunctionCall* callSqrt = new FunctionCall("sqrt", minus);
Variable* var = new Variable("var");
BinaryOperation* mult = new BinaryOperation(var, BinaryOperation::MUL,
callSqrt);
FunctionCall* callAbs = new FunctionCall("abs", mult);

CopySyntaxTree CST;
Expression* newExpr = callAbs->transform(&CST);
```

Часть 2.

Копирование AST можно было реализовать, например, просто добавив виртуальный метод clone, похожий на метод evaluate. Но используя Visitor, через единственный метод transform

мы можем реализовать не только копирование.

Задание 2. (5 баллов) В этой задаче вам необходимо реализовать сворачивание констант в дереве (constant folding). Например, у нас есть выражение (точнее, дерево, описывающее это выражение) `abs(var * sqrt(32.0 - 16.0))`, на выходе мы должны получить дерево для следующего выражения `abs(var * 4.0)`, т. е. подвыражение `sqrt(32.0 - 16.0)` было вычислено. Для того, чтобы определить, что выражение (Expression) на самом деле является числом (Number), используйте оператор `dynamic_cast`. Все промежуточные узлы дерева, которые вы создали, нужно освободить.

```
struct Transformer;
struct Number;
struct BinaryOperation;
struct FunctionCall;
struct Variable;

struct Expression
{
    virtual ~Expression() { }
    virtual double evaluate() const = 0;
    virtual Expression *transform(Transformer *tr) const = 0;
};

struct Transformer // pattern Visitor
{
    virtual ~Transformer() { }
    virtual Expression *transformNumber(Number const *) = 0;
    virtual Expression *transformBinaryOperation(BinaryOperation const *) = 0;
    virtual Expression *transformFunctionCall(FunctionCall const *) = 0;
    virtual Expression *transformVariable(Variable const *) = 0;
};

struct Number : Expression
{
    Number(double value);
    double value() const;
    double evaluate() const;
    Expression *transform(Transformer *tr) const;

private:
    double value_;
};

struct BinaryOperation : Expression
{
    enum {
        PLUS = '+',
        MINUS = '-',
        DIV = '/',
        MUL = '*'
    };
    BinaryOperation(Expression const *left, int op, Expression const *right);
    ~BinaryOperation();
    double evaluate() const;
    Expression *transform(Transformer *tr) const;
    Expression const *left() const;
    Expression const *right() const;
};
```

```
int operation() const;

private:
    Expression const *left_;
    Expression const *right_;
    int op_;
};

struct FunctionCall : Expression
{
    FunctionCall(std::string const &name, Expression const *arg);
    ~FunctionCall();
    double evaluate() const;
    Expression *transform(Transformer *tr) const;
    std::string const &name() const;
    Expression const *arg() const;

private:
    std::string const name_;
    Expression const *arg_;
};

struct Variable : Expression
{
    Variable(std::string const name);
    std::string const &name() const;
    double evaluate() const;
    Expression *transform(Transformer *tr) const;

private:
    std::string const name_;
};

/**
 * реализуйте все необходимые методы
 * если считаете нужным, то можете
 * заводить любые вспомогательные
 * методы
 */
struct FoldConstants : Transformer
{
    Expression *transformNumber(Number const *number)
    {
        // ваш код
    }

    Expression *transformBinaryOperation(BinaryOperation const *binop)
    {
        // ваш код
    }

    Expression *transformFunctionCall(FunctionCall const *fcall)
    {
        // ваш код
    }

    Expression *transformVariable(Variable const *var)
    {

```

```
    // ваш код  
}  
};
```

Проверьте работу созданной структуры, например, на следующем коде

```
Number* n32 = new Number(32.0);  
Number* n16 = new Number(16.0);  
BinaryOperation* minus = new BinaryOperation(n32, BinaryOperation::MINUS,  
n16);  
FunctionCall* callSqrt = new FunctionCall("sqrt", minus);  
Variable* var = new Variable("var");  
BinaryOperation* mult = new BinaryOperation(var, BinaryOperation::MUL,  
callSqrt);  
FunctionCall* callAbs = new FunctionCall("abs", mult);  
  
FoldConstants FC;  
Expression* newExpr = callAbs->transform(&FC);
```

Часть 3. Индивидуальные задания.

Вариант 1 (10 баллов). Используя паттерн проектирования Visitor составьте структуру классов Персона → (Студент, Профессор), Visitor → (Сессия, Каникулы), в которой есть следующие виды деятельности: в Сессию Студент сдает экзамены, а Профессор принимает экзамены, соответственно, в Каникулы Студент отдыхает, Профессор уходит в отпуск. Продемонстрируйте работу паттерна.

Доп. задания:

(5 баллов) В существующую иерархию надо добавить новую операцию «Пройти аттестацию». Напишите необходимые изменения в коде, продемонстрируйте работу паттерна.

(5 баллов) Необходимо добавить в существующую иерархию класс Аспирант. Внесите необходимые изменения в код, продемонстрируйте работу паттерна.

Вариант 2 (10 баллов). Используя паттерн Strategy запрограммируйте следующую ситуацию. Имеется класс (или шаблон класса) Array (массив), который (массив) можно заполнять различным образом: используя стратегию заполнения массива из файла, стратегию заполнения значениями элементов по умолчанию. Продемонстрируйте работу паттерна.

Доп. задания:

(5 баллов) В существующую иерархию необходимо добавить стратегию, позволяющую заполнить массив целых чисел случайными значениями в диапазоне от 0 до randMax. Внесите необходимые изменения в код, продемонстрируйте работу паттерна.

Вариант 3 (10 баллов) Есть два семейства продуктов: открытки (PostCard) и подарки (Present). Они бывают двух видов: новогодние (NYCard, NYPresent) и на день рождения (BDCard, BDPresent). Семейства новогодних и ДР комплектов создаются при помощи соответствующих фабрик (NYFactory, BDFactory), являющихся наследниками абстрактной фабрики (AbstractFactory). Напишите предложенную иерархию классов, используя паттерн AbstractFactory.

Доп. задания:

(5 баллов) В программу решили добавить новый продукт — торт (Cake), различая его виды: новогодний и торт на день рождения (NYCake, BDCake). Внесите необходимые изменения в код, продемонстрируйте работу паттерна.

(5 баллов) В программу решили добавить новый праздник — 8 марта (т.е. теперь доступны открытки «с 8 марта» и подарки на 8 марта). Внесите необходимые изменения в код, продемонстрируйте работу паттерна.

Вариант 4 (10 баллов) Разработайте классы PointDecart (точка в декартовых координатах) и PointPolar (точка в полярных координатах), которые содержат в себе следующую информацию: поля — координаты точки, методы — конструктор, методы чтения координат, методы установки значений координат, метод вывода значений координат на консоль. Напишите адаптер ОБЪЕКТА, который приспособит PointPolar к PontDecart. Пр продемонстрируйте работу паттерна.

Вариант 5 (10 баллов) Разработайте классы PointDecart (точка в декартовых координатах) и PointPolar (точка в полярных координатах), которые содержат в себе следующую информацию: поля — координаты точки, методы — конструктор, методы чтения координат, методы установки значений координат, метод вывода значений координат на консоль. Напишите адаптер КЛАССА, который приспособит PointPolar к PontDecart. Пр продемонстрируйте работу паттерна.

Вариант 6 (10 баллов) Используя паттерн AbstractFactory запрограммируйте следующую задачу. Есть два вида застройки: высокой и низкой этажности. Высокоэтажная застройка подразумевает создание жилых домов (9-20 этажей) и офисов (12-20 этажей). Низкоэтажная застройка подразумевает создание жилых коттеджей (1-2 этажа) и школ (1-4 этажа). Напишите соответствующие иерархии классов (или структур) для жилых (дома, коттеджи) и нежилых помещений (офисы, школы), напишите соответствующие фабрики для низкоэтажной и высокоэтажной застройки. Пр продемонстрируйте применение фабрик в программе.

Вариант 7 (15 баллов) Дан рецепт (точнее, состав блюда): шампиньоны (500г), креветки (100г), сметана (30г), сыр (20г), укроп и пряности (30г). Для каждого продукта известны (на 100г продукта) энергетическая ценность, количество белков, жиров и углеводов, а также стоимость 100 г продукта.

Шампиньоны — 27 кКал, 4,4г белки, 1г жиры, 0,2г углеводы, 20 руб

Креветки — 83 кКал, 18г белки, 1 г жиры, 0г углеводы, 50 руб

Сметана — 120 кКал, 3,3г белки, 10 г жиры, 3,3 г углеводы, 14 руб 40 коп

Сыр — 345 кКал, 25 г белки, 25 г жиры, 0 г углеводы, 70 руб

Укроп и пряности — 37 кКал, 3.3 г белки, 0 г жиры, 7 г углеводы, 20 руб

Напишите иерархию классов «Продукты» и класс (или структуру) «Рецепт блюда».

При помощи паттерна Visitor подсчитайте:

1. калорийность блюда
2. количество белков в блюде
3. количество жиров в блюде

4. количество углеводов в блюде
5. стоимость блюда

Вариант 8 (10 баллов) При помощи паттерна Strategy реализуйте следующие способы печати одномерного массива:

1. печать по n элементов в строке через пробел
2. печать по одному элементу в строке

Продемонстрируйте работу паттерна.

Вариант 9 (10 баллов) Разработайте иерархию классов Суп → (Борщ, Тыквенный суп), ВтороеБлюдо → (ЖаренаяКартошка, Котлета). При помощи паттерна AbstractFactory разработайте классы Диета → (МяснаяДиета, ВегетарианскаяДиета). Продемонстрируйте работу паттерна.

Доп. задания:

(5 баллов) Добавьте в этот паттерн новое блюдо — десерт. Внесите необходимые изменения в код, продемонстрируйте работу паттерна

(5 баллов) Добавьте в этот паттерн новую диету - кефирную (первым блюдом для нее будет окрошка, вторым — пудинг, третьим — мороженое или йогурт).

Вариант 10 (10 баллов) Пусть есть класс «Связный список», в котором определены операции добавления-удаления в начало, добавления-удаления элемента в конец списка, проверки, не пуст ли список. Напишите паттерн Adapter (адаптер класса), который адаптирует класс «Связный список» к интерфейсу класса «Стек».

Вариант 11 (10 баллов) Пусть есть класс «Связный список», в котором определены операции добавления-удаления в начало, добавления-удаления элемента в конец списка, проверки, не пуст ли список. Напишите паттерн Adapter (адаптер объекта), который адаптирует класс «Связный список» к интерфейсу класса «Стек»

Вариант 12 (10 баллов) Разработайте иерархию классов «Занятие» («Лекция», «Лабораторное занятие», «Семинар»). В каждом занятии можно учесть количество часов, отведенных на него, и максимальный балл, который дается за одно занятие. При помощи паттерна Visitor подсчитайте суммарное количество часов по предмету (во всех видах занятий) и максимальный балл.

Вариант 13 (10 баллов) Разработайте иерархию классов: «Средства разработки» (конкретные классы «Delphi», «Visual Studio»), «Операционные системы» («Win XP», «Win 8»). Delphi может функционировать только под управлением ОС WinXP (семейство «старых» лицензированных программ), Visual Studio – только под управлением Win 8 (семейство «новых» лицензированных программ). Реализуйте данную ситуацию при помощи паттерна Abstract Factory. Добавьте также еще один продукт (например, компиляторы — Borland для «старых» лицензий, Visual — для «новых»), и семейство средств с открытой лицензией (ОС Linux, IDE NetBeans, компиляторы MinGW).

Вариант 14 (10 баллов) Реализуйте при помощи паттерна Strategy для класса «Массив» (а лучше, для шаблона класса «Массив») стратегии сортировки

1. выборкой
2. пузырьком

Продемонстрируйте работу стратегий.

Вариант 15 (10 баллов) Разработайте иерархию классов «Ингредиент» (например, конкретными ингредиентами могут быть продукты: картошка, свекла, лук, майонез, рыба...) Для каждого продукта указывается название и калорийность на 100г, а также количество продукта. Из продуктов составляется рецепт блюда (например, продукты могут туда входить списком, или массивом, или вектором...) При помощи паттерна Visitor узнайте состав ингредиентов рецепта, а также посчитайте калорийность блюда.

Вариант 16 (10 баллов) Разработайте следующие иерархии классов: рыбы (например, барбусы, карпы, сомы), моллюски (большие и малые улитки), водоросли. Для каждого объекта учитывается название. Все объекты составляют аквариум (рыбы, моллюски и водоросли могут входить туда списком, или вектором, или как-то иначе). При помощи паттерна Visitor узнайте состав аквариумных обитателей, подсчитайте количество особей в аквариуме.

Вариант 17 (10 баллов) Есть класс «Оплата за телефон», в котором учитывается, сколько было исходящих вызовов, сколько времени занял каждый из них, и учитывается вид тарифа. При помощи паттерна Strategy реализуйте следующее поведение в классе:

1. тариф с абонентской платой
2. посекундная тарификация
3. поминутная тарификация

Вариант 18 (10 баллов) Разработайте следующие иерархии классов: «Обувь» («Кроссовки», «Туфли»), «Одежда» («Спортивный костюм», «Смокинг»). Кроссовки и спортивный костюм, очевидно, относятся к категории спортивной одежды, а смокинг и туфли — к категории вечерних нарядов. При помощи паттерна Abstract Factory реализуйте указанную ситуацию.

Доп. задания:

(5 баллов) Что нужно видоизменить в программе, чтобы добавить туда категорию ежедневной одежды? Внесите изменения в код, продемонстрируйте применение паттерна.

(5 баллов) Что нужно видоизменить в программе, если надо добавить новый продукт, например, сумки? Внесите изменения в код, продемонстрируйте применение паттерна.

Вариант 19 (10 баллов) Напишите класс «Рабочее время», в котором учитывается длительность рабочего дня в астрономических часах. Напишите класс «Нагрузка преподавателя», где указывается количество академических часов, которые он должен отработать в течение учебного года. Реализуйте адаптер класса, который адаптирует класс «Рабочее время» к интерфейсу класса «Нагрузка преподавателя». Один академический час = 45 астрономическим минутам. Длительность рабочего года преподавателя примите за 44 учебные недели (для обычного сотрудника рабочий год длится 48 недель).

Вариант 20 (20 баллов) Напишите классы шкал измерения температур («по Цельсию», «по Фаренгейту», «температура по Кельвину»). Напишите адаптеры объекта, которые переведут температуру из градусов Цельсия в Кельвины ($T \text{ градусов К} = T \text{ градусов С} + 273$),

из градусов Фаренгейта в Кельвины ($T \text{ градус С} = (T \text{ градусов Ф} - 32) * 5/9 + 273$),
из градусов Цельсия в Фаренгейты ($T \text{ градусов Ф} = T \text{ градусов С} * 9/5 + 32$),
из Фаренгейта в Цельсии ($T \text{ градус С} = (T \text{ градусов Ф} - 32) * 5/9$). Продемонстрируйте работу адаптеров.

Вариант 21 (10 баллов) Реализуйте иерархии классов «Книга» («Сказки», «Учебник»), «Фильмы» («Мультфильм», «Документальный фильм»). Сказки и мультфильмы относятся к категории детских товаров, учебники и документальные фильмы — к категории учебных товаров. При помощи паттерна Abstract Factory реализуйте описанную ситуацию.

Доп. задания:

(5 баллов) Что надо будет изменить в программе, чтобы добавить туда еще один товар (например, ресурсы интернет?) Внесите изменения в код, продемонстрируйте работу паттерна.

(5 баллов) Что надо будет изменить в программе, чтобы добавить категорию (например, развлекательные товары — детектив, фантастический фильм)? Внесите изменения в код, продемонстрируйте работу паттерна.